

SEISMOGUARD

AI-Powered Seismic Early Warning System

COMPLETE PROJECT BRIEFING

Full Technical Documentation for Supervisor Presentation

Project Name	SeismoGuard -- IoT Seismic Early Warning
Hardware	ESP32 Microcontroller + MPU6050 Accelerometer
Communication	UDP over Wi-Fi (port 4210)
Middleware	Python 3 -- UDP Listener + MySQL Writer
Backend	PHP 8 -- REST API (api.php)
Frontend	HTML5 + CSS3 + Vanilla JavaScript
Database	MySQL -- quakesense / seismic_data
Charts	Chart.js v4 + Annotation Plugin
Total Files	4 core files: .ino, .py, .php, .html

Contents

1	What Is This Project?	4
1.1	The Big Picture — One Sentence Per Layer	4
1.2	Data Journey — From Sensor to Screen	4
2	File 1 — Arduino Firmware (earth-AI_ino.ino)	6
2.1	Hardware Components	6
2.2	Configuration Variables	7
2.3	Every Function Explained	7
2.3.1	setup() — Runs Once at Power-On	7
2.3.2	readMPU(ax, ay, az) — Reads Raw Sensor Data	8
2.3.3	loop() — Runs Forever, Every 100 ms	8
2.3.4	triggerAlarm() — Called During EARTHQUAKE!	8
2.4	The Detection Logic	8
3	File 2 — Python Middleware (receiver.py)	10
3.1	Configuration Variables	10
3.2	Every Function Explained	10
3.2.1	connect_db() — Opens MySQL Connection	10
3.2.2	parse_packet(raw) — Parses the ESP32 Message	10
3.2.3	save_to_db(db, cursor, status, diff_value) — Writes to MySQL	11
3.2.4	main() — The Entry Point and Main Loop	11
4	File 3 — PHP Backend (api.php)	12
4.1	Request and Response Format	12
4.2	Step-by-Step Code Walkthrough	12
4.2.1	Step 1 — HTTP Headers	12
4.2.2	Step 2 — Database Connection	12
4.2.3	Step 3 — Pagination Calculation	12

4.2.4	Step 4 — Total Count Query	13
4.2.5	Step 5 — Records Query	13
4.2.6	Step 6 — Stats Query	13
4.2.7	Step 7 — JSON Response	13
4.3	Error Handling Summary	14
5	File 4 — Web Dashboard (index.html)	15
5.1	Technologies Used	15
5.2	The Four Tabs	16
5.3	CSS Design System	16
5.4	JavaScript State Variables	17
5.5	Complete JavaScript Function Reference	17
5.5.1	updateClock()	17
5.5.2	switchTab(tabId)	17
5.5.3	playAlert(type)	18
5.5.4	showAlert(diffVal) / dismissAlert()	18
5.5.5	animateCount(element, target)	18
5.5.6	updateRing(status)	18
5.5.7	updateGauge(diffVal)	18
5.5.8	hexToRgba(hex, alpha)	18
5.5.9	initWaveform() and draw()	19
5.5.10	initChart()	19
5.5.11	fetchData() — The Core Function	19
5.5.12	fetchHistory(page)	20
5.5.13	changePage(dir) / goToPage(page)	20
5.5.14	runDemo()	20
5.5.15	exportToCSV()	21
6	Database — quakesense / seismic_data	22

6.1	Table Structure	22
6.2	Record Statistics from Your Data	22
7	How to Run the Complete System	23
7.1	Required Folder Structure	23
7.2	Start-Up Order	23
7.3	Troubleshooting	24
8	Questions Your Supervisor May Ask	25

1. What Is This Project?

SeismoGuard is a real-time IoT-based earthquake early warning system built as an academic capstone project. It uses a physical hardware sensor to detect ground vibrations, processes the data through multiple software layers, and displays live results on a professional web dashboard.

The system is split into four layers that work together as a pipeline — hardware detects motion, Python stores it, PHP serves it, and the web browser displays it.

1.1 The Big Picture — One Sentence Per Layer

Layer	What It Does in Plain Words
ESP32 + MPU6050	A small computer with a motion sensor — it feels vibrations and decides if it is a WARNING or EARTHQUAKE
Python Receiver	Sits on a laptop and listens for messages from the ESP32 over Wi-Fi, then saves them to the database
PHP API (api.php)	When the web page asks for data, PHP reads the database and sends it back as structured JSON text
HTML Dashboard	The web page the user sees — graphs, live waveform, stats, and alerts, updating every 3 seconds

1.2 Data Journey — From Sensor to Screen

Listing 1: Complete data pipeline from ESP32 to browser

```
1 STEP 1 -> ESP32 reads accelerometer (MPU6050) 10 times per
    second
2         calculates how much ground moved (diff value)
3         decides: NORMAL, WARNING, or EARTHQUAKE!
4         sends a UDP message over Wi-Fi
5
6 STEP 2 -> Python script receives UDP message on port 4210
7         parses: "WARNING | D:0.4512"
8         saves status + diff_value + timestamp to MySQL
```

```
9
10 STEP 3 -> Browser asks api.php every 3 seconds
11           PHP reads latest rows from MySQL
12           returns JSON: { records:[...], stats:{...} }
13
14 STEP 4 -> JavaScript reads the JSON
15           updates waveform, gauge, stats, table, chart
16           triggers alert if new earthquake detected
```

2. File 1 — Arduino Firmware (earth-AI_ino.ino)

This is the firmware that runs directly on the ESP32 microcontroller chip, written in C++ using the Arduino framework. This code never stops running — it loops continuously, reading the sensor, making decisions, and sending data every 100 ms.

2.1 Hardware Components

Component	Purpose in This Project
MPU6050	Measures acceleration on 3 axes (X, Y, Z) — detects physical vibration
LCD (16x2)	Shows current status and diff value on the hardware device itself
LED (Pin 2)	Turns ON solid when EARTHQUAKE! status is triggered
Buzzer (Pin 13)	Sounds 5 rapid alarm beeps during EARTHQUAKE! alert
Wi-Fi	Connects to iPhone hotspot and sends UDP packets to the laptop

2.2 Configuration Variables

Variable	What It Stores
<code>ssid / password</code>	Wi-Fi credentials to connect to the laptop's hotspot network
<code>udpAddress</code>	"172.20.10.2" — IP address of the laptop running <code>receiver.py</code>
<code>udpPort = 4210</code>	Port number that both ESP32 and Python use to communicate
<code>MPU = 0x68</code>	I2C hardware address of the MPU6050 sensor on the Wire bus
<code>threshold = 0.8</code>	Critical diff value above which the counter increments toward EARTHQUAKE!
<code>quakeCounter</code>	Running count — increments on high motion, decrements when calm
<code>filteredDiff</code>	Smoothed version of diff using the low-pass filter formula

2.3 Every Function Explained

2.3.1 `setup()` — Runs Once at Power-On

Runs exactly once when the ESP32 powers on or resets:

1. Starts Serial at 115200 baud for USB debugging
2. Initialises I2C bus (`Wire.begin()`) for MPU6050 and LCD communication
3. Sets LED and Buzzer pins as OUTPUT
4. Starts the LCD and displays "QuakeSense AI / Starting..."
5. Connects to Wi-Fi — tries up to 20 times with 0.5 s waits between each
6. Wakes up the MPU6050 by writing 0 to its power management register (0x6B)

2.3.2 readMPU(ax, ay, az) — Reads Raw Sensor Data

Communicates with MPU6050 over I2C and fills three float variables with acceleration values on each axis. The raw 16-bit integers are divided by 16384.0 to convert to g-force units ($1.0\text{ g} = \text{Earth's gravity}$). Reads 14 bytes total: 6 for acceleration, 2 for temperature (skipped), 6 for gyroscope (skipped).

2.3.3 loop() — Runs Forever, Every 100 ms

The heart of the system. Each cycle:

1. Calls `readMPU()` to get fresh acceleration values
2. Calculates $\text{accMagnitude} = \sqrt{ax^2 + ay^2 + az^2}$ (total movement vector)
3. Calculates $\text{diff} = |\text{accMagnitude} - \text{prevMagnitude}|$ (change from last reading)
4. Applies **low-pass filter**: $\text{filteredDiff} = 0.7 \times \text{old} + 0.3 \times \text{new}$ (smooths noise)
5. Updates `quakeCounter`: +1 if $\text{filteredDiff} > \text{threshold}$, -1 if calm
6. Assigns status: $\text{quakeCounter} > 6 \Rightarrow \text{EARTHQUAKE!}$, $> 2 \Rightarrow \text{WARNING}$, else `NORMAL`
7. Builds message string: e.g. `"WARNING | D:0.4512"`
8. Sends UDP packet to the laptop's IP and port
9. Updates the LCD with current status and diff value

2.3.4 triggerAlarm() — Called During EARTHQUAKE!

Overrides LCD with `"!!! ALERT !!!"` and `"EVACUATE NOW"`. Loops 5 times turning the buzzer ON for 150 ms then OFF for 150 ms.

2.4 The Detection Logic

A simple threshold (if $\text{diff} > 0.8$, alarm) would trigger from any single bump. The counter system requires *sustained* motion: the threshold must be exceeded for more than 6 consecutive readings (≈ 0.6 seconds) before declaring EARTHQUAKE! This dramatically reduces false alarms.

Listing 2: quakeCounter decision logic (Arduino C++)

```
1 filteredDiff > 0.8    ->  quakeCounter++    // sustained strong
```

```
    motion
2  filteredDiff <= 0.8 -> quakeCounter--    // settling down
3
4  if (quakeCounter > 6) status = "EARTHQUAKE!" // 6+
    consecutive spikes
5  if (quakeCounter > 2) status = "WARNING"      // building up
6  else                                     status = "NORMAL"      // calm
```

3. File 2 — Python Middleware (receiver.py)

This is the middleware — a Python script running on the same laptop as the web server. It bridges the ESP32 hardware and the MySQL database by listening for Wi-Fi messages and saving the important ones (WARNING and EARTHQUAKE!) to the database.

3.1 Configuration Variables

Variable	Value and Purpose
UDP_IP = "0.0.0.0"	Listen on ALL network interfaces — accepts packets from any source IP
UDP_PORT = 4210	Must exactly match <code>udpPort</code> in the Arduino code
DB_HOST = "localhost"	Database is on the same machine as this script
DB_NAME = "quakesense"	MySQL database name — must match what XAMPP has set up

3.2 Every Function Explained

3.2.1 `connect_db()` — Opens MySQL Connection

Uses the `mysql-connector-python` library to establish a database connection. Returns a connection object on success or `None` on failure. The `main()` function checks for `None` and halts — it would be pointless to receive data with nowhere to store it.

3.2.2 `parse_packet(raw)` — Parses the ESP32 Message

The ESP32 sends plain text like `"WARNING | D:0.4512"`. This function:

1. Splits by `"|"` → `["WARNING ", " D:0.4512"]`
2. Strips whitespace from both parts
3. Splits the second part by `":"` → `["D", "0.4512"]`
4. Converts `"0.4512"` to a Python `float`

5. Returns the tuple ("WARNING", 0.4512)

On any parse error it prints a diagnostic and returns `None`; the main loop skips that packet safely.

3.2.3 `save_to_db(db, cursor, status, diff_value)` — Writes to MySQL

Executes a parameterised `INSERT` statement. The `%s` placeholders prevent SQL injection. Calls `db.commit()` to permanently save the row. Returns `True` on success, `False` on error.

Listing 3: Parameterised `INSERT` used by `save_to_db()`

```
1 INSERT INTO seismic_data (status, diff_value, recorded_at)
2 VALUES (%s, %s, %s)
3 -- Example: ("WARNING", 0.4512, "2026-04-22 16:08:55")
```

3.2.4 `main()` — The Entry Point and Main Loop

1. Creates a UDP socket (`AF_INET = IPv4`, `SOCK_DGRAM = UDP`)
2. Sets `SO_REUSEADDR = 1` to fix Windows Error 10048 (port still in use on restart)
3. Binds the socket to port 4210 — begins listening for packets
4. Calls `connect_db()` and stops if it returns `None`
5. Enters an infinite `while` loop; `recvfrom(1024)` blocks until a packet arrives
6. Decodes received bytes to a UTF-8 string and calls `parse_packet()`
7. Only saves to MySQL if status is "WARNING" or "EARTHQUAKE!"
8. Prints every event to the terminal for live monitoring

The `try/except KeyboardInterrupt` block handles `Ctrl+C` gracefully, closing the cursor, database connection, and socket cleanly.

The ESP32 sends 10 packets per second — 600 per minute. Saving `NORMAL` events would create tens of thousands of rows per hour with no useful information. Only anomalies matter for the dashboard, so `NORMAL` packets are printed to the terminal but never inserted into the database.

4. File 3 — PHP Backend (api.php)

This is the bridge between the database and the web page. When JavaScript calls `fetch("api.php")`, the XAMPP Apache server executes this PHP script. It reads MySQL and responds with a structured JSON object. It runs at `http://localhost`.

4.1 Request and Response Format

HTTP Request	What Is Returned
<code>GET api.php</code>	Page 1 (latest 50 records) + full stats
<code>GET api.php?page=2</code>	Records 51–100 + same full stats
<code>GET api.php?page=5</code>	Records 201–250 + same full stats
Out-of-range page number	Automatically clamped to the last valid page

4.2 Step-by-Step Code Walkthrough

4.2.1 Step 1 — HTTP Headers

- `Content-Type: application/json` — tells the browser the response is JSON
- `Access-Control-Allow-Origin: *` — allows any origin to read this response (CORS)

4.2.2 Step 2 — Database Connection

Creates a `mysqli` connection. If `$conn->connect_error` is set, outputs a JSON error and calls `exit` immediately — no further code runs.

4.2.3 Step 3 — Pagination Calculation

Listing 4: Pagination offset calculation

```
1 $perPage = 50;
```

```

2 $page      = isset($_GET["page"]) ? max(1, (int)$_GET["page"])
    : 1;
3 $offset    = ($page - 1) * $perPage;
4 // page=1 -> offset=0    (rows 1-50)
5 // page=2 -> offset=50   (rows 51-100)
6 // page=3 -> offset=100  (rows 101-150)

```

`max(1, ...)` prevents negative or zero page numbers. `(int)` casts the input safely to an integer.

4.2.4 Step 4 — Total Count Query

Runs `SELECT COUNT(*)` to find the total number of rows. Used to calculate total pages: `totalPages = ⌈totalRecords/50⌉`. The requested page is then clamped to this maximum.

4.2.5 Step 5 — Records Query

Fetches exactly 50 rows from the calculated offset, ordered `DESC` by `recorded_at`. Uses `DATE_FORMAT()` to guarantee a clean `"YYYY-MM-DD HH:MM:SS"` timestamp string.

4.2.6 Step 6 — Stats Query

Aggregates across the *entire* table (not just the current page). Uses `IFNULL(SUM(...), 0)` to return 0 instead of `NULL` when the table is empty. All values are cast to proper PHP types (`int`, `float`) before encoding.

4.2.7 Step 7 — JSON Response

Listing 5: Structure of `api.php` JSON output

```

1 {
2   "records": [
3     { "id":"1884", "status":"WARNING",
4       "diff_value":"0.08", "recorded_at":"2026-04-22 16:22:00
5       " },
6     ...
7   ],
8   "stats":      { "total":1884, "warnings":1866,

```

```
8         "earthquakes":18, "max_diff":2.52 },
9     "page":          1,
10    "total_pages":   38,
11    "total_records":1884
12 }
```

4.3 Error Handling Summary

Error Situation	PHP Response	What Frontend Sees
Cannot connect to MySQL	JSON error + exit	Red message: DB connection failed
COUNT query fails	JSON error + exit	Red message: Count query failed
Records query fails	JSON error + exit	Red message: Records query failed
Stats query fails	JSON error + exit	Red message: Stats query failed
Page out of range	Clamp, continue	Last valid page displayed

5. File 4 — Web Dashboard (index.html)

The entire frontend is a single HTML file containing HTML structure, CSS styles, and JavaScript all in one place. No external frameworks (React, Vue, Angular) are used. It connects to `api.php` every 3 seconds and updates every visual element with fresh sensor data.

5.1 Technologies Used

Technology	Why It Is Used
HTML5	Page structure — panels, tables, buttons, canvas elements
CSS3 + Variables	Dark theme styling, animations, responsive grid layout
Vanilla JavaScript	All interactivity — fetching data, updating the DOM, animations
Chart.js v4 (CDN)	Draws the line chart in the Analytics tab
Annotation Plugin	Adds WARNING and EARTHQUAKE dashed threshold lines to the chart
Google Fonts (CDN)	Orbitron (headings), Space Mono (data labels), Syne (body)
Canvas 2D API	The live scrolling seismograph waveform animation
Web Audio API	Generates the earthquake alarm sound without any audio files
Fetch API	Makes HTTP GET requests to <code>api.php</code> every 3 seconds

5.2 The Four Tabs

Tab	Contents
Dashboard	Seismo Hero (waveform + ring + gauge), 4 stat cards, latest 5 events table
Live Graph	Chart.js line chart of last 50 events with threshold lines + analytics stats sidebar
Event Log	Paginated table — 50 records per page, slider + prev/next navigation, CSV export
Project Info	Architecture diagram, detection thresholds, project description for presentations

5.3 CSS Design System

All colors and fonts are defined as CSS custom properties in `:root{}` — changing one variable updates the entire page.

Variable	Value	Used For
<code>-teal</code>	<code>#00f5d4</code>	NORMAL state — waveform, borders, primary accent
<code>-amber</code>	<code>#f5a623</code>	WARNING state — gauges, badges, stat cards
<code>-red</code>	<code>#ff3860</code>	EARTHQUAKE! state — alerts, danger indicators
<code>-bg</code>	<code>#04080f</code>	Main page background (very dark navy)
<code>-surface</code>	<code>rgba(8,16,28,</code>	Panel and card backgrounds (semi-transparent)
<code>-font-head</code>	Orbitron	Logo, status values, large numbers
<code>-font-mono</code>	Space Mono	All data values, timestamps, badges, labels
<code>-font-body</code>	Syne	Paragraph text and descriptions

5.4 JavaScript State Variables

Variable	What It Tracks
<code>globalRecords</code>	Full records array from last API response — used by CSV export
<code>chartInstance</code>	Holds the Chart.js object — guards against creating duplicate charts
<code>soundEnabled</code>	<code>true</code> = play alarm sounds, <code>false</code> = muted
<code>prevQuakeCount</code>	Earthquake count from the last fetch — alerts only fire when this increases
<code>waveAmplitude</code>	Current waveform pixel height, smoothly animated toward <code>waveTargetAmp</code>
<code>waveTargetAmp</code>	Target amplitude: 6 (NORMAL), 20 (WARNING), 38 (EARTHQUAKE!)
<code>waveColor</code>	Hex color of the waveform: teal / amber / red based on status
<code>historyPage</code>	Which page of the Event Log is currently displayed
<code>historyTotal</code>	Total number of pages available from the database

5.5 Complete JavaScript Function Reference

5.5.1 `updateClock()`

Called every 1000 ms via `setInterval`. Uses the browser's new `Date()` to update three DOM elements: the header clock (HH:MM:SS), the header date, and the footer timestamp. Runs entirely in the browser with no server request.

5.5.2 `switchTab(tabId)`

Removes `.active` from all `.tab-content` divs and all tab buttons, then adds it to only the selected tab. Extra: if switching to "analytics", calls `chartInstance.update()` to redraw the chart; if switching to "history", clears the notification dot and calls `fetchHistory()` to load paginated data.

5.5.3 playAlert(type)

Generates a 4-note descending alarm (EARTHQUAKE!) or 2-note warning tone using the Web Audio API. Creates `OscillatorNode` objects set to "sawtooth" wave type, schedules four frequencies (880, 660, 440, 330 Hz) spaced 120 ms apart, and fades each out quickly. No audio file needed — the sound is synthesised mathematically. Wrapped in `try/catch` because browsers block audio until the user has interacted with the page.

5.5.4 showAlert(diffVal) / dismissAlert()

`showAlert()` simultaneously: displays the red pulsing border overlay, shows the earthquake popup modal with shake animation, and calls `playAlert("quake")`. `dismissAlert()` reverses both. Also triggered by the Escape keyboard shortcut.

5.5.5 animateCount(element, target)

Animates a number counting smoothly from its current displayed value to the target. Uses `setInterval` at 25 increments per second until the target is reached. For example, jumping from 50 to 99 events visually counts: 50, 53, 56, ..., 99.

5.5.6 updateRing(status)

Controls the SVG arc in the Status Ring using the `stroke-dasharray/stroke-dashoffset` technique. Circle circumference = $2\pi \times 55 \approx 345$ px. Dashoffset to 345 hides the arc; 0 shows the full circle. Sets it to $345 \times (1 - \text{pct})$ where pct is 28% for NORMAL, 60% for WARNING, 100% for EARTHQUAKE!. CSS `transition` animates the change smoothly over 1 second.

5.5.7 updateGauge(diffVal)

Same SVG technique as `updateRing` for the intensity gauge (radius 50, circumference 314 px). Maps diff value to 0–100% using `max=1.2`. Colors: teal below 0.3, amber 0.3–0.8, red above 0.8. CSS transition animates the arc over 1.2 seconds.

5.5.8 hexToRgba(hex, alpha)

Converts a hex color like "#00f5d4" to "`rgba(0, 245, 212, 0.53)`". Required because the Canvas 2D API's `addColorStop()` needs `rgba()` for alpha values — 8-digit hex colors are not reliably supported in canvas gradient context across all browsers.

5.5.9 `initWaveform()` and `draw()`

Sets up the `<canvas>` and runs an infinite 60 fps loop via `requestAnimationFrame`. Each frame:

1. Clears the canvas
2. Draws faint grid lines (oscilloscope background look)
3. Draws a dashed center baseline
4. Interpolates `waveAmplitude` toward `waveTargetAmp` by 4% per frame (linear interpolation)
5. Draws the wave using a compound sine:

$$y = \sin(t) + 0.35 \sin(2.1t) + 0.2 \sin(0.5t) + 0.1 \sin(3.7t)$$

Multiple frequencies combined to simulate realistic seismic noise

6. Draws a "NOW" cursor marker at 75% across the canvas
7. Advances `wavePhase` by 0.06 to scroll the wave leftward

5.5.10 `initChart()`

Creates the Chart.js line chart, called only once (guarded by `if (!chartInstance)`). Key settings:

- `backgroundColor` — gradient callback from 25% opacity at top to transparent at bottom
- `pointBackgroundColor` — callback colors each point: red ≥ 0.8 , amber ≥ 0.3 , teal otherwise
- `update("none")` — skips animation on refresh to avoid jarring redraws every 3 seconds
- Annotation plugin adds dashed threshold lines at $y = 0.3$ and $y = 0.8$ with colour labels
- `Chart.register()` wrapped in `try/catch` — the CDN already auto-registers the plugin; calling it again would throw “already registered” and crash the entire fetch chain

5.5.11 `fetchData()` — The Core Function

Runs on page load and every 3 seconds. The complete sequence:

1. `fetch("api.php")` — HTTP GET request to the PHP backend
2. Checks `r.ok` (HTTP 200) — throws on error
3. Calls `r.text()` to get raw response (not `.json()` directly)
4. Manually calls `JSON.parse()` in `try/catch` — if PHP returned an error page instead of JSON, the raw error text is shown in the table so the developer can read it
5. Updates stat cards with `animateCount()`
6. Updates status text, diff value, waveform labels
7. Calls `updateRing()` and `updateGauge()`
8. Sets `waveTargetAmp` and `waveColor` based on latest status
9. If `newQuake > prevQuakeCount` — new earthquake detected — calls `showAlert()` and `showNotif()`
10. Builds and inserts table rows using `buildRow()`
11. Calls `initChart()` on first run, then updates chart dataset
12. Calls `updateAnalytics()` for the sidebar stats

The `.catch()` handler shows a visible error in the table instead of failing silently to the console.

5.5.12 `fetchHistory(page)`

A separate fetch function for the Event Log tab only. Calls `api.php?page=N`. On success: updates pagination controls (slider range, current page label, prev/next button states, total record count). Updates `globalRecords` so CSV export works on the current page.

5.5.13 `changePage(dir) / goToPage(page)`

`changePage(+1)` or `changePage(-1)` moves one page forward or back. `goToPage(n)` is bound to the range slider's `oninput` event, jumping directly to the selected page. Both guard against going below 1 or above `historyTotal`.

5.5.14 `runDemo()`

Designed for classroom presentations when hardware is not connected. Immediately sets all visual components to EARTHQUAKE! state (ring, gauge, waveform, modal) and plays the alarm. After 6 seconds, `setTimeout()` automatically resets everything to NORMAL. Purely visual — no database interaction.

5.5.15 `exportToCSV()`

Downloads the current page as a CSV file with no server interaction. Creates a `Blob` (in-memory file) from a JavaScript string, generates a temporary URL with `URL.createObjectURL()`, programmatically clicks a hidden `<a>` element to trigger the browser's download dialog, then removes the element.

6. Database — quakesense / seismic_data

6.1 Table Structure

Column	Type	Purpose
id	INT AUTO_INCREMENT	Unique row ID, auto-assigned by MySQL
status	VARCHAR(20)	Exactly "WARNING" or "EARTHQUAKE!" — NORMAL is never stored
diff_value	FLOAT	Filtered acceleration difference, e.g. 0.4512
recorded_at	DATETIME	Auto-filled with date and time of INSERT

6.2 Record Statistics from Your Data

Metric	Value	Notes
Total records	1884+	As of April 22, 2026 test session
WARNING records	~1866	Approximately 99% of all stored records
EARTHQUAKE! records	~18	High-intensity events only
Max diff_value	2.52	Peak motion recorded
Min diff_value	0.08	Near-zero reading after event settles
Pages in Event Log	38+	At 50 records per page

7. How to Run the Complete System

7.1 Required Folder Structure

Listing 6: File placement for XAMPP

```
1 C:\xampp\htdocs\capstone\  
2   |-- index.html      (web dashboard)  
3   +-- api.php         (PHP backend)  
4  
5 (anywhere on your laptop)  
6   +-- receiver.py     (run this in a terminal)
```

7.2 Start-Up Order

1. Open **XAMPP Control Panel** → click **Start** on Apache → click **Start** on MySQL
2. Open a terminal → navigate to folder containing `receiver.py` → run `python receiver.py`
3. Power on the ESP32 device — it connects to Wi-Fi and begins transmitting
4. Open **Chrome** → type: `http://localhost/capstone/index.html`
5. Dashboard loads and begins refreshing every 3 seconds automatically

7.3 Troubleshooting

Problem	Cause	Solution
No data loads	Opened via file://	Use <code>http://localhost/capstone/index.html</code>
DB connection error	Wrong credentials	Check <code>\$user</code> , <code>\$pass</code> , <code>\$dbname</code> in <code>api.php</code>
Python Error 10048	Port already in use	Run: <code>taskkill /F /IM python.exe</code>
ESP32: WiFi Failed	Wrong credentials in .ino	Update <code>ssid</code> and <code>password</code> , re-upload
No data in Python	Wrong laptop IP in .ino	Check your IP and update <code>udpAddress</code>
Chart tab blank	Plugin registration error	Already fixed in current <code>index.html</code>

8. Questions Your Supervisor May Ask

UDP is connectionless and faster — no handshake required. Since the ESP32 sends 10 packets per second and occasional packet loss is acceptable (missing one reading in ten does not affect accuracy), UDP is ideal. TCP would add connection management overhead to the embedded code unnecessarily.

The `quakeCounter` mechanism requires *sustained* motion. The filtered diff must exceed 0.8 for more than 6 consecutive readings (≈ 0.6 seconds) before triggering EARTHQUAKE!. A single knock or vibration spike only adds 1 to the counter. The low-pass filter ($0.7 \times \text{old} + 0.3 \times \text{new}$) further smooths instantaneous spikes.

`fetchData()` stores `prevQuakeCount` after every API call. On the next call, if `newQuake > prevQuakeCount` the count increased since the last check — the alert fires. This ensures alerts trigger only once per new event, not on every 3-second refresh.

The waveform runs at 60 fps via `requestAnimationFrame` — it is a Canvas animation, not a direct sensor stream. The amplitude and colour are updated from the API every 3 seconds. Between updates, the wave continues scrolling at its last known amplitude, giving a professional live-monitor appearance.

The **DEMO MODE** button triggers a full simulated earthquake alert — ring, gauge, waveform, modal, and sound all activate simultaneously, then auto-reset after 6 seconds. Specifically designed for presentations where live hardware may not be present.

A single-file approach minimises deployment complexity — just place it in the web root. No build tools, no npm, no compilation step required. All functionality uses browser-native APIs (Canvas, Web Audio, Fetch, CSS Grid), demonstrating core web technologies directly rather than relying on abstractions.

The Web Audio API is a browser-built-in sound synthesis engine. `playAlert()` creates `OscillatorNode` generators set to "sawtooth" wave type, schedules four descending frequencies (880, 660, 440, 330 Hz) 120 ms apart, and fades each one out. The sound is generated mathematically in real time — no MP3 or WAV file is needed.